



Online Metric Matching on the Line with Recourse

Nicole Megow¹ · Lukas Nölke¹

Received: 22 June 2022 / Accepted: 3 February 2025 / Published online: 1 March 2025
© The Author(s) 2025

Abstract

In online metric matching on the line, n requests appear one by one and have to be matched immediately and irrevocably to a given set of servers, all located on the real line. The goal is to minimize the sum of distances between the requests and their assigned servers. The best known online algorithm achieves a competitive ratio of $\Theta(\log n)$, leaving a gap to the best-known lower bound of $\Omega(\sqrt{\log n})$. In this work, we approach the problem in a recourse model where online decisions can be partially revised, allowing for the reassignment of previously matched edges. In contrast to the traditional online setting, we show that with an amortized recourse budget of $O(\log n)$, we can obtain an $O(1)$ -competitive algorithm for online metric matching on the line. This is one of the first non-trivial results for metric matching with recourse. Additionally, for so-called alternating instances, where no more than one request lies between two servers, we achieve a near-optimal result. Specifically, we give a simple algorithm that is $(1 + \varepsilon)$ -competitive and reassigns any request at most $O(\frac{1}{\varepsilon^2})$ times. This special case is particularly noteworthy, as a lower bound of $\Omega(\log n)$, constructed using such instances, applies to a broad class of online algorithms, including all deterministic algorithms studied in the literature.

Keywords Minimum-cost matching in bipartite graphs · Recourse · Competitive analysis · Online algorithms

Mathematics Subject Classification 90C27 · 05C85

Nicole Megow and Lukas Nölke have contributed equally to this work.

An extended abstract with parts of the results has been published in the Proceedings of the International Workshop on Approximation Algorithms for Combinatorial Optimization, APPROX 2020, [23].

✉ Nicole Megow
nmegow@uni-bremen.de

Lukas Nölke
noelke@uni-bremen.de

¹ Faculty of Mathematics and Computer Science, University of Bremen, Bremen, Germany

1 Introduction

Matching problems are among the most fundamental problems in combinatorial optimization with great importance in both, theory and applications. In the (bipartite) metric matching problem, we are given a complete bipartite graph $G = (R \cup S, E)$ with positive edge cost $c(e) = c(s, r)$, for $e = (s, r) \in E$. Elements of R and S are called *requests* and *servers*, respectively, with $n := |R| \leq |S|$. A *matching* $M \subseteq E$ is a set of pairwise non-incident edges. A matching is called *complete* if every request in R is matched to a server in S , i.e., if it is incident to exactly one edge of M . The metric matching problem asks to compute a complete matching of minimum cost, where the *cost* of a matching M is given by $c(M) := \sum_{e \in M} c(e)$. When all information is given in advance, the optimum can be computed efficiently, e.g., by using the Hungarian Method as has been shown by Kuhn and Yaw [20].

In the *online setting*, however, the set of requests is not known a priori. Requests arrive online, one by one, and have to be matched immediately and irrevocably to an unmatched server. As we cannot hope to find an optimal matching under these restrictions, we use standard competitive analysis to evaluate the performance of algorithms. An online matching algorithm is said to be α -*competitive* if it computes for any instance a matching M with $c(M) \leq \alpha \cdot \text{OPT}$, where OPT denotes the cost of an optimal matching. The *competitive ratio* of an algorithm is the smallest α for which it is α -competitive. Independently, Kalyanasundaram and Pruhs [17], and Khuller, Mitchell, and Vazirani [19] showed that for arbitrary edge costs, the competitive ratio of any online algorithm is unbounded. They also developed the same deterministic algorithm, *Permutation*, which is $(2n - 1)$ -competitive for metric costs and showed that this is optimal for deterministic online algorithms. A remarkable recent result by Nayyar and Raghvendra [26] is a fine-grained analysis of an online algorithm based on *t-net cost* [28] showing a competitive ratio of $O(\mu(G) \log^2 n)$, where $\mu(G)$ denotes the maximum ratio of the length of the minimum TSP tour and the weighted diameter of a subset of G . This result is particularly interesting as it relates structural parameters of the metric space to the performance guarantee.

For a long time, the online metric matching problem has resisted all attempts for achieving an $O(1)$ -competitive algorithm, even for special metric spaces such as the line. In *online metric matching on the line*, the edge costs are induced by a line metric; that is, we identify each vertex of G with a point on the real line and the cost of an edge between a request and a server equals their distance on the line. The competitive ratio of the aforementioned algorithm in [26] is then $O(\log^2 n)$, as $\mu(G) = 2$. Subsequently, Raghvendra [29] improved this to $\Theta(\log n)$. Only very recently, Peserico and Scquizzato [27] showed that no online algorithm can achieve a competitive ratio better than $\Omega(\sqrt{\log n})$. Thus, the quest for finding an $O(1)$ -competitive algorithm, even a randomized one, is indeed hopeless, at least when restricted to the strict online setting.

In this paper, we relax the strict irrevocability requirement and consider online metric matching on the line with *recourse*. In the recourse model, we allow for limited modifications to past decisions, thereby relaxing the standard online model. Specifically, at any point, we may delete a set of edges $\{(r_i, s_i)\}_i$ of the current matching and rematch the requests r_i to different (free) servers.

Online optimization with recourse is an alternative model to standard online optimization and has gained increasing attention as a framework for *beyond worst-case analysis*. While an $O(1)$ -competitive algorithm is impossible for online metric matching even on the line, an interesting question arises: how much recourse is needed to achieve a constant competitive ratio. Beyond its theoretical appeal, recourse also aligns well with practical applications where limited revisions to prior decisions are feasible and beneficial.

Clearly, if recourse is unrestricted, it becomes possible to simulate an optimal offline algorithm, and the online nature of the problem disappears. We define an algorithm to have an *amortized recourse budget* β if it rematches requests at most βn times in total. The key question for online *metric* matching on the line is determining whether it is possible to maintain an $O(1)$ -competitive solution with bounded recourse—that is, with a sublinear recourse budget.

1.1 Our Results

We answer this question to the affirmative and give non-trivial results for the online metric matching problem with recourse. We show that with limited recourse, one can indeed maintain a constant competitive solution on the line.

Theorem 1 *The online metric matching problem on the line admits an $O(1)$ -competitive algorithm with amortized recourse budget $O(\log n)$.*

Our algorithm builds on the t -net cost algorithm by Raghvendra [28, 29]; details follow later. It has the property of smoothly interpolating between an $O(\log n)$ -competitive online solution (without any recourse) and an $O(1)$ -approximate offline solution (with possibly large recourse). Our algorithm can further be adapted to allow for a range of different cost-recourse tradeoffs. We observe that our analysis is asymptotically optimal.

Theorem 2 *For any function $f : \mathbb{N} \rightarrow \mathbb{N}$ with $1 \leq f(n) \leq \log n$, there is an algorithm for online metric matching on the line that is $O(f(n))$ -competitive and has an amortized recourse budget of $O(\frac{\log n}{f(n)})$.*

Additionally, we investigate a special class of instances, called *alternating* instances, where between any two requests on the line there is at least one server. This class is interesting as the quite strong lower bound of $\Omega(\log n)$, given by Antoniadis, Fischer, and Tönnis [2], holds already on such simple instances. While it does not hold for arbitrary instances, the bound is true for all known deterministic online algorithms *without recourse*. For alternating instances, we present a more direct and near-optimal algorithm with a scalable performance-recourse trade-off.

Theorem 3 *For alternating instances of online metric matching on the line, there is a $(1 + \varepsilon)$ -competitive algorithm that reassigns each request $O(\frac{1}{\varepsilon^2})$ times.*

While the algorithm is quite simple, the proof requires a clever charging scheme that exploits the special structure of optimal solutions on alternating instances. We

observe that a large number of recourse actions for a specific request implies large edges in the optimal solution elsewhere on the line.

As a byproduct we give a simple analysis of (a variant of) the algorithm in the traditional online setting without recourse. We show that it has a competitive ratio of $O(\min\{\log n, \log \Delta\})$ for instances with alternating requests on the line, where Δ is the ratio between the largest and shortest request-server distance, also called the aspect ratio of the metric. This result refines the competitive ratio $\Theta(\log n)$ of the currently best known online algorithm [29].

Remark Simultaneously and independently of our work, Gupta, Krishnaswamy and Sandeep [15] obtained a similar result for online metric matching with recourse on the line. Their algorithm builds on the $O(n)$ -competitive Permutation algorithm [17, 19] and adapts it for the recourse setting. On the line, this is done by first matching edges according to Permutation and then asymmetrically applying recourse to arcs (r, s) of the current matching that overlap in a certain way. Both, their algorithm and analysis are completely different from ours. They further obtain a more general $O(\log n)$ -competitive algorithm with amortized recourse $O(\log n)$ for arbitrary metrics. Note that Permutation in fact coincides with the t -net cost algorithm for $t = 1$.

1.2 Further Related Work

Extensive literature is devoted to online bipartite matching problems. The *maximum matching* variant is quite well understood. For the unweighted setting, asking for a maximum-cardinality matching, Karp, R. Vazirani, and U. Vazirani [18] gave optimal deterministic and randomized algorithms with competitive ratio 2 and $e/(e - 1)$, respectively. The weighted maximization setting does not admit bounded guarantees in general, but intensive research investigates models with additional assumptions; see, e.g., the survey by Mehta [25].

The online metric matching problem, asking for a *minimum-cost matching*, is much less understood. In arbitrary metric spaces, the best possible deterministic algorithm has a competitive ratio of $(2n - 1)$ [17, 19]. Randomization in the algorithms decisions admits better algorithms; there is an $O(\log^2 n)$ -competitive randomized algorithm due to Bansal et al. [3]. On the line, no such improvement on the deterministic result by randomization is known; the competitive ratio of $O(\log n)$ is the best-known result for both, deterministic and randomized, algorithms [14, 29]. It remains a large gap between to the best-known lower bound of $\Omega(\sqrt{\log n})$ by Peserico and Squizzato [27].

Interestingly, Gairing and Klimm [9] showed that when assuming a random order of request arrivals instead of an adversarial arrival order, the natural Greedy algorithm that matches an arriving request to the closest *free*, i.e., unmatched, server is n -competitive for general metric spaces. Furthermore, the online t -net cost algorithm is $O(\log n)$ -competitive [28] in this case. Very recently, Gupta et al. [12] gave an $O((\log \log \log n)^2)$ -competitive algorithm in the model with online known i.i.d. arrivals.

Recourse models received quite some attention in the past decade. In particular, maintaining an online cardinality-maximal bipartite matching with recourse was studied extensively; see, e.g., [1, 4–7, 10, 30] and references therein. Bernstein et al. [5] showed that the 2-competitive greedy algorithm uses amortized $O(n \log^2 n)$ reassignments, leaving a small gap to the lower bound of $\Omega(n \log n)$. In contrast, for the min-cost variant, it remained a challenging question whether recourse can improve upon the competitive ratio. Even on the line, it remained open whether and how recourse can improve the bound of $O(\log n)$ [29].

The following two models address other types of matching with recourse. In a setting motivated by scheduling, several requests can be matched to the same server and the goal is to minimize the maximum number of requests assigned to a server. Gupta et al. [13] achieve an $O(1)$ -competitive ratio with amortized $O(n)$ edge reassignments. A quite different two-stage robust model has been proposed recently by Matuschke et al. [22]. In a first stage, one must compute a perfect matching on a given graph and in a second stage a batch of $2k$ new nodes appears which must be incorporated into the first-stage solution to maintain a low-cost matching by reassigning only few edges. For matching on the line, they give an algorithm that maintains a 10-approximate matching reassigning $2k$ edges.

Recourse in online optimization has also been explored for other minimum-cost problems, although it has been studied less extensively than for maximization problems. A prominent example is the online minimum Steiner tree problem [11, 16, 21, 24]. In this problem, an $O(1)$ -competitive algorithm can be achieved with just one edge reassignment per iteration [11], whereas in the standard online setting without recourse, the competitive ratio is $\Omega(\log n)$.

The recourse model has some relation to *dynamic algorithms*. Instead of minimizing the number of past decisions that are changed (recourse), the dynamic model focuses on the running time to implement this change (*update time*). A full body of research exists on maximum (weighted) bipartite matching; we refer to the nice survey by Chaudhuri et al. [8]. We are not aware of any results for maintaining a matching of minimum cost.

2 Preliminaries

A path P is called *alternating* with respect to a matching M , if every other edge in P is contained in M . An alternating path is called *augmenting* with respect to M if it starts and ends at vertices not covered by M . A common method for increasing the cardinality of an existing matching M is to *augment* along an augmenting path P , resulting in a larger matching $\tilde{M}$ given by the symmetric difference¹ $M \oplus P$. There may be a choice between different augmenting paths; typically, a path of minimum cost (w.r.t. some metric) is selected. Recently, Raghvendra [28] introduced the following metric. For $t > 1$, the *t-net cost* of a path P w.r.t. a matching M is defined as

¹ For two sets X, Y , their symmetric difference is given by $X \oplus Y := (X \cup Y) \setminus (X \cap Y)$. For matchings M_1, M_2 , their symmetric difference $M_1 \oplus M_2$ consists of disjoint paths and cycles, whose edges are alternating between M_1 and M_2 .

$$\phi_i^M(P) := t \cdot c(P \setminus M) - c(P \cap M) = t \cdot c(P \cap \tilde{M}) - c(P \cap M). \quad (1)$$

Our algorithm maintains three matchings: the *recourse matching* M , the actual output of the algorithm, and two auxiliary matchings based on (online and offline versions of) the t -net cost algorithm [28], namely, the *offline matching* M^* and the *online matching* M' . While M^* is a near-optimal offline matching that possibly requires a large amount of recourse, M' is an online matching that is $O(\log n)$ -competitive [29] but uses no recourse. We describe how M^* and M' are obtained based on the above cost function; see also [26, 28, 29]. By speaking of the matching M_i , M_i^* or M'_i , we refer to the state of the respective matching after serving the i -th request.

Upon arrival of the i -th request r_i , the *offline* t -net cost algorithm constructs the offline matching M_i^* by augmenting M_{i-1}^* along an alternating path P_i of minimum t -net cost w.r.t. M_{i-1}^* . That is, $M_i^* := M_{i-1}^* \oplus P_i$. By definition, this path starts at r_i and ends at a free server, which we denote by s_i . While this procedure may require a large amount of recourse, the resulting matching has been shown to have bounded cost.

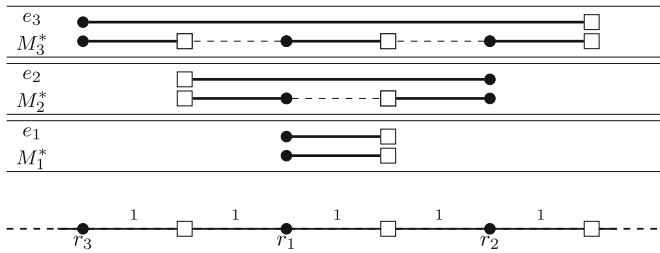
Lemma 4 (Raghvendra [28]) *For any $t > 1$, we have $c(M_i^*) \leq t \cdot \text{OPT}_i$ for all $i \in \{1, 2, \dots, n\}$, where $\text{OPT}_i := c(M_i^{\text{OPT}})$ denotes the cost of an optimal offline matching M_i^{OPT} of the first i requests.*

For constructing the online matching M'_i without changing previous matching decisions (no recourse), it may not be possible to augment along a path. Instead, the online t -net cost algorithm maintains M^* as an auxiliary matching and constructs M'_i by directly connecting the end points r_i and s_i of the augmenting path P_i . That is, $M'_i := M'_{i-1} \cup \{(r_i, s_i)\}$, cf. Fig. 1a. In particular, M'_i and M_i^* utilize the same sets of servers.

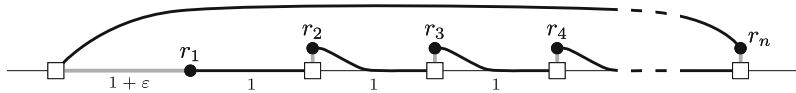
Intuitively, in putting a higher weight on edges that would be added to M^* during augmentation, the parameter t in the t -net cost function discourages the offline t -net cost algorithm from choosing long paths for augmentation (w.r.t. actual costs, not in the t -net-cost metric). This allows for a trade-off between minimizing the cost of the underlying offline matching and minimizing the connection cost in M' with the latter occurring in a greedy fashion. Looking at the extremal cases, this becomes even more clear.

When $t = 1$, the offline t -net cost algorithm is in fact equivalent to the Hungarian Method [20] which computes an optimal offline solution. The corresponding online matching, however, is that of the Permutation algorithm [17, 19] and has a competitive ratio of $\Theta(n)$, see [17, 19] and the lower bound instance in Fig. 1a.

In contrast, as t tends to infinity, the algorithm increasingly favors augmenting via paths of shorter length, making its behavior more similar to that of the greedy online algorithm, which matches a request upon arrival to the nearest free server. The competitive ratio in this case is $\Theta(n)$, see [28] and Fig. 1b. Interestingly though, when $t = 3$, the t -net cost algorithm has a competitive ratio of $O(\log n)$ [29]. Furthermore, it is worth noting that t -net cost algorithm with $t \rightarrow \infty$ achieves in the random order arrival model the best possible competitive ratio of $O(\log n)$ [28].



(a) An (alternating) instance where the t -net cost algorithm, with $t = 1$, has a competitive ratio of $\Omega(n)$. At time i , the optimal offline matching M^* increases in cost by 1, while M' increases in cost by $2i - 1 = c(e_i)$. Thus $\text{OPT} = n$ and $c(M') = \Omega(n)$.



(b) An instance where the t -net cost algorithm, with $t \rightarrow \infty$, has a competitive ratio of $\Omega(n)$. For $1 < i < n$, the augmenting path from r_i to the next free server on the left has cost $t \cdot (1 + \varepsilon) - (i - 1)$ and to the next free server on the right cost t . With $\varepsilon > \frac{1}{t}(i - 1) \rightarrow 0$, the t -net cost algorithm augments to the right resulting in a cost of $\Omega(n)$, while $\text{OPT} = 1 + \varepsilon$.

Fig. 1 Lower bound instances for the t -net cost algorithm and extremal choices of t

3 A Constant-Competitive Algorithm with Bounded Recourse

In this section, we prove our main results, Theorems 1 and 2, by describing the corresponding algorithms.

We begin by giving a high-level overview of our algorithm for Theorem 1. It exploits the properties of the t -net cost algorithm by carefully balancing between the offline matching M^* and the online matching M' , simultaneously bounding competitive ratio and recourse budget. On a high level, this is done as follows. When a request arrives, we match it as in M' and locally group it with other recent requests into *blocks* that correspond to intervals on the line. Matching requests as in the online matching increases the total cost but requires zero recourse. A structural result, Lemma 13, allows us to bound this increase in cost, but only for a set of blocks whose corresponding intervals are pairwise disjoint. Thus, when new blocks are created, the requests in old intersecting blocks need to be rematched according to M^* causing a local update, which we call a *recourse step*. During such a recourse step, the changes in M^* caused by the arrivals of the requests in the respective old blocks are applied simultaneously. This eliminates any redundant recourse action that would have occurred when updating M^* repeatedly, for each arriving request. Intuitively, blocks can be seen as input buffers for M^* that temporarily match requests as in M' and facilitate an efficient update of M^* in large local batches. The underlying structure of the blocks guarantees that recourse steps affect only the corresponding portion of the line. To prevent the recourse steps from causing too many reassignments, we additionally incorporate an edge freezing scheme for low-cost edges that keeps the overall cost low.

3.1 A First Attempt: Balancing Between M^* and M' Without Freezing

We start our pursuit of a constant competitive algorithm with bounded recourse by exploring the idea of balancing between M^* and M' , and show that this first strategy does indeed yield a constant competitive algorithm. While we will see that this strategy uses potentially a lot of recourse, the algorithm we develop here serves as the basis of our final algorithm satisfying Theorem 1. The latter simply requires an additional freezing scheme targeting low-cost edges, which we present in the following subsection.

3.1.1 Definitions and Notation

We classify requests according to the structure of intervals that describe where on the line the t -net cost algorithm has already searched for free servers, following the ideas from Raghvendra [29]. While the definitions used in that work differ slightly, as we elaborate below, they serve essentially the same purpose. Throughout the paper, we visualize the real line as a horizontal line from $-\infty$ on the left to ∞ on the right and use according vocabulary, e.g., by saying a server s is left of a request r , or simply $s < r$. We also assume, without loss of generality, that at any point on the line, there is at most one point of $R \cup S$.

Define the *search interval* of a request r_i as the open interval $\bar{I}_i = (s_i^L, s_i^R)$, where s_i^L and s_i^R denote points on the line farthest to the left and right of r_i , respectively, and reachable from r_i with t -net-cost at most $\phi_t^{M_{i-1}^*}(P_i)$. We will show in Lemma 10 that one of s_i^L, s_i^R is the server s_i (which r_i is matched to in M'), while the other may not necessarily be a point of $R \cup S$. For the purpose of this definition, we think of it as a (virtual) server. That is, we ask the question “If point p was a server in S , would we be able to reach it via an augmenting path of t -net cost at most $\phi_t^{M_{i-1}^*}(P_i)$?”. In other words, $\bar{I}_i$ is the convex hull of all points on the line, reachable from r_i via an augmenting path of t -net cost (strictly) less than $\phi_t^{M_{i-1}^*}(P_i)$.

Define the *aggregate search interval* of r_i to be the maximal (open) interval I_i which contains r_i and is a subset of $\bigcup_{j \leq i} \bar{I}_j$. Intuitively, the set $\bigcup_{j \leq i} \bar{I}_j$ consists of the (disjoint) portions of the line, which the t -net cost algorithm has already considered, up to time i , in its search for free servers; the interval I_i is simply the connected portion containing r_i . See Fig. 2 for an illustration. By definition, the portions of the line constituting $\bigcup_{j \leq i} \bar{I}_j$ grow monotonously (and possibly merge while doing so). Thus, the aggregate search intervals inhibit a laminar structure as detailed in the following observation.

Observation 5 *Whenever $i < j$, then either $I_i \cap I_j = \emptyset$ or $I_i \subseteq I_j$.*

We say an aggregate search interval I_i is of *level* k , if its length satisfies $(1 + \varepsilon)^{k-1} \leq |I_i| < (1 + \varepsilon)^k$; we write $\ell(I_i) = k$. Throughout the remainder of this section, we set $t = 3$ and $\varepsilon = \frac{1}{32t}$. Further, two aggregate search intervals are said to belong to the same *block*, if they intersect with each other and are of same level. If the aggregate search intervals of a block are not contained in some interval of a higher level, then

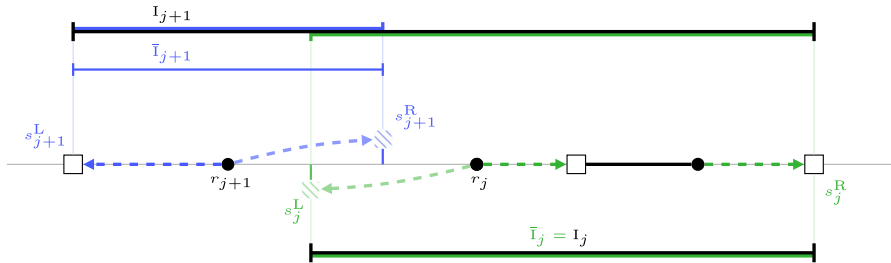


Fig. 2 Construction of search intervals and aggregate search intervals of requests r_j and r_{j+1} . The points s_j^L and s_{j+1}^R (hatched) are points on the line that do not lie in S . We think of them as virtual servers for the purpose of defining $\bar{I}_j$ and $\bar{I}_{j+1}$ respectively

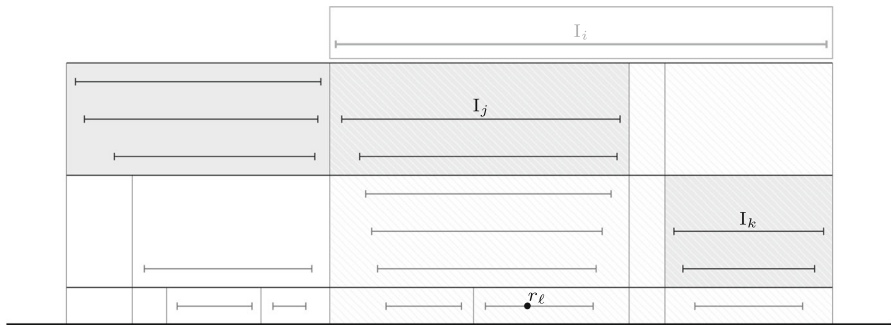


Fig. 3 Illustration of a typical block structure. On arrival of r_i , its aggregate search interval I_i creates a new top block (active). Requests in the hatched area are now all inactive. We observe that on I_i , locally, $M_{i-1}^* = M_h^*$ for $j, k \leq h < i$

this block is said to be a *top block*. We note that the top blocks partition the line into portions that are compatible with the structure of M^* . Specifically, we will see that, at all times, the edges of M^* (and M') are fully contained in some top block. Intuitively, this holds since the portion of the line corresponding to a top block is the convex hull of points considered for the searches for a free server originating from requests inside the block, so no edge of M^* can possibly lead outside the block. A typical block-structure is depicted in Fig. 3.

Based on these structures, we dynamically classify requests as follows. When the aggregate search interval of a request belongs to a top block then this request is said to be *active*. Once this no longer holds, we call it *inactive*. Note that due to the definition of aggregate search intervals, any arriving request is initially active. And once a request becomes inactive, it will remain inactive indefinitely. When considering the attributes active, inactive, or membership in a block, (or later on *frozen*), we identify a request r_i , the edge $e_i = \{r_i, s_i\} \in M'$ and the interval I_i . For instance, we say r_i is of level k and belongs to a certain block, when this holds for I_i , or, e_i is active when this is the case for r_i .

3.1.2 Algorithm Description

Our algorithm, summarized as Algorithm 1 produces the *recourse matching* M by assigning requests according to their classification as either active or inactive. Specifically, active requests are matched in M exactly as in the online matching M' , and inactive requests are matched as in the offline matching M^* .

While the online matching M' does not change but only gets revealed over time, the offline matching M^* may change its structure drastically at any point causing a lot of recourse. Active requests act as an input buffer for M^* , it is updated only periodically and locally when a new top block appears and renders previously active requests inactive. We call this a *recourse step*.

The algorithm then consists of the following two steps that are executed whenever a new request r_i arrives.

Matching r_i Upon arrival of r_i , label it active and match it according to the online matching M' , that is, via the edge $e_i = (r_i, s_i)$.

Recourse step If there is no $j < i$ such that $I_j \subseteq I_i$ and $\ell(I_j) = \ell(I_i)$, the arrival of r_i produces a new top block and may render a number of previously active requests inactive, see Fig. 3. Reassign all requests $r_j \in I_i$ except r_i to the same server to which they are matched in M_{i-1}^* .

Algorithm 1 : Balancing M' and M^* without Freezing

Upon arrival of the i -th request r_i :		$\triangleright r_i$ matched in M' via $e_i = (r_i, s_i)$
Matching r_i		
1: match r_i to s_i as in M'		$\triangleright r_i$ active
Recourse step		
2: if there is an active $r_j \in I_i$ with $\ell(I_j) < \ell(I_i)$ then		$\triangleright I_i$ creates new top block
3: for $r_j \in I_i$ with $j \neq i$ do		
4: reassign r_j to the same server as in M_{i-1}^*		$\triangleright r_i$ inactive

3.1.3 Algorithm Analysis

We start by noting that Algorithm 1 indeed produces a matching after each iteration. In Step 1, r_i can be matched to s_i , since both are not yet matched by definition of the t -net cost algorithm. Therefore, the edge (r_i, s_i) also does not interfere in a possible recourse step due to the arrival of r_i , as it is not contained in M_{i-1}^* .

Observation 6 *At the end of each iteration, Algorithm 1 produces a feasible matching M_i .*

Bounding the competitive ratio We first observe, that the cost of inactive edges can be easily bounded using Lemma 4.

Corollary 7 *The total cost of inactive edges is at most $t \cdot \text{OPT}$.*

To bound the cost of active edges, we build on the analysis of Raghvendra [29]. We refine his technical propositions and perform a slightly more fine-grained analysis.

Instead of simultaneously bounding the cost of all blocks of the same level, we argue more generally on the cost of any set of pairwise disjoint blocks at possibly different levels, where two blocks are said to be *disjoint* if their corresponding maximal aggregate search intervals do not intersect. In particular, we are interested in bounding the cost contribution of the disjoint set of top blocks. We show the following result.

Lemma 8 *For $t = 3$, the cost of all active edges of M is bounded by $O(\text{OPT})$.*

Corollary 7 and Lemma 8 together imply a constant competitive ratio.

Corollary 9 *Algorithm 1 has a competitive ratio of $O(1)$.*

For the sake of completeness, we give a proof of Lemma 8, including the adaptations from [29]. We start by discussing our definition of search intervals. While this definition was motivated by intuition and practicality (specifically with the proof of Lemma 19 below in mind), it describes intervals different from the “search intervals” in [29]. However, we show that our definition of aggregate search intervals coincides with Raghvendra’s definition of *intervals of a cumulative search region*, which we denote by C_i . We may therefore use the corresponding results from [29].

Lemma 10 *For a request r_i , the corresponding aggregate search interval I_i coincides with the “interval of a cumulative search region” C_i defined in [29]. Further, for the search interval $\bar{I}_i = (s_i^L, s_i^R)$, we have $s_i \in \{s_i^L, s_i^R\}$.*

Proof To see that the first claim holds, note that the intervals C_i and I_i are constructed the same way. That is, they are built by taking the union of all known search intervals (for the respective definition) and choosing of the resulting new intervals that which contains the considered request. We describe the definition of search intervals in [29], which, to avoid confusion, we call *dual intervals*. The t -net cost algorithm maintains dual values $y^i : S \cup R \rightarrow \mathbb{R}^+$ satisfying $y_s^i + y_r^i = c(s, r)$ if $(s, r) \in M_i^*$ and $y_s^i + y_r^i \leq t \cdot c(s, r)$ otherwise. Additionally, duals of free requests or servers are zero. When a request r_i arrives, a shortest t -net-cost path P_i is found and the duals of all vertices in the search tree, which is partitioned by sets $A_i \subseteq S$ and $B_i \subseteq R$, are updated before augmentation so that the dual constraints on P_i are tight. This is true for both the augmenting paths P_i^L and P_i^R that are used to reach s_i^L and s_i^R , respectively. The dual interval of a request r_i is defined as $\text{interior}(\bigcup_{r \in B_i} \text{cspan}(r, i))$, where $\text{cspan}(r, i)$ denotes the interval $[r - \frac{y_{\max}^i(r)}{t}, r + \frac{y_{\max}^i(r)}{t}]$ on the real line and $y_{\max}^i(r)$ the highest dual weight assigned to r until time i .

Raghvendra [28] shows, that the dual constraints on P_i^L and P_i^R are tight before augmentation. Therefore, we obtain our search intervals $\bar{I}_i$ from the respective dual interval by replacing $y_{\max}^i(r)$ with the dual weight y_r^i of r before augmentation along P_i . Hence, a search interval is contained in the corresponding dual interval and therefore $I_i \subseteq C_i$. At the same time, dual weights of requests can only be increased when the request is contained in some B_i or reduced right after an augmentation but remain otherwise unchanged. Thus, the maximal value $y_{\max}^i(r)$ is attained right before an augmentation, in which case the request is part of some P_h^L or P_h^R . Using again that the dual weights are tight here, this implies $C_i \subseteq I_i$. Therefore, the intervals I_i and C_i coincide and we may use the respective results from [29].

The second claim follows directly from Lemma 6 in [29], which states that there are no free servers in the dual interval. As seen above, search intervals are contained in dual intervals, so this statement is true for our definition as well. And lastly, since clearly $s_i \in [s_i^L, s_i^R]$, it follows that $s_i \in \{s_i^L, s_i^R\}$. $\square$

We continue with a technical result that allows us to bound the cost contribution of edges of M' whose length is large when compared to the t -net cost of the corresponding augmenting path. Formally, we call an edge e_i of the online matching M' or the corresponding augmenting path P_i *short*, if $c(P_i) \leq \frac{4}{t-1} \phi_t^{M_{i-1}^*}(P_i)$ and *long* otherwise. Raghvendra bounds the cost of all long edges by that of all short ones. We give the following more general statement that allows us to consider a consecutive set of edges instead. Note that the additional additive term of $2(t+1) \cdot \text{OPT}_{j-1}$ when compared to the original statement in [29] disappears again, when considering the entire matching, as then $\text{OPT}_{j-1} = \text{OPT}_0 = 0$.

Lemma 11 *Let $P_j, P_{j+1}, \dots, P_k$ be consecutive augmenting paths that are used in the offline t -net cost algorithm and $e_j, e_{j+1}, \dots, e_k$ the corresponding edges in M' . Then*

$$\sum_{i: e_i \text{ long}} c(e_i) \leq \left(4 + \frac{4}{t-1}\right) \sum_{i: e_i \text{ short}} c(e_i) + 2(t+1) \cdot \text{OPT}_{j-1}.$$

Proof To prove this lemma, we adapt the approach of [29, Lemma 2] for our more general setting. Consider two consecutive matchings M_{i-1}^* and M_i^* , and recall from Eq. (1) that the t -net cost of the corresponding augmenting path P_i is given by $\phi_t^{M_{i-1}^*}(P_i) = t \cdot c(P_i \setminus M_{i-1}^*) - c(P_i \cap M_{i-1}^*)$. The difference in cost of the two matchings can then be expressed as

$$\begin{aligned} \frac{t+1}{2} (c(M_i^*) - c(M_{i-1}^*)) &= \frac{t+1}{2} (c(P_i \setminus M_{i-1}^*) - c(P_i \cap M_{i-1}^*)) \\ &= \phi_t^{M_{i-1}^*}(P_i) - (t-1) \cdot c(P_i \setminus M_{i-1}^*) + \frac{t-1}{2} (c(P_i \setminus M_{i-1}^*) - c(P_i \cap M_{i-1}^*)) \\ &= \phi_t^{M_{i-1}^*}(P_i) - \frac{t-1}{2} (c(P_i \setminus M_{i-1}^*) + c(P_i \cap M_{i-1}^*)) \\ &= \phi_t^{M_{i-1}^*}(P_i) - \frac{t-1}{2} \cdot c(P_i). \end{aligned}$$

Summing from j to k , and using the fact that $\text{OPT}_{j-1} \leq \text{OPT}_k \leq c(M_k^*)$ as well as $c(M_{j-1}^*) \leq t \cdot \text{OPT}_{j-1}$, we get

$$\sum_{i=j}^k \left(\phi_t^{M_{i-1}^*}(P_i) - \frac{t-1}{2} \cdot c(P_i) \right) = \frac{t+1}{2} \sum_{i=j}^k (c(M_i^*) - c(M_{i-1}^*)) \geq \frac{1-t^2}{2} \cdot \text{OPT}_{j-1}. \quad (2)$$

Denote by L and H the sets of augmenting paths with index between j and k that are long and short, respectively. Then, Eq. (2) can be rewritten as

$$\sum_{P_i \in H} \phi_t^{M_{i-1}^*}(P_i) \stackrel{(2)}{\geq} \frac{t-1}{2} \sum_{P_i \in H} c(P_i) + \sum_{P_i \in L} \left(\frac{t-1}{2} \cdot c(P_i) - \phi_t^{M_{i-1}^*}(P_i) \right) + \frac{1-t^2}{2} \cdot \text{OPT}_{j-1}$$

$$\geq \sum_{P_i \in L} \phi_t^{M_{i-1}^*}(P_i) + \frac{1-t^2}{2} \cdot \text{OPT}_{j-1}, \quad (3)$$

where the last inequality uses the fact that $\sum_{P_i \in H} c(P_i)$ is non-negative and that for a long path P_i we have $c(P_i) \geq \frac{4}{t-1} \phi_t^{M_{i-1}^*}(P_i)$. Using again non-negativity of $\sum_{P_i \in H} c(P_i)$, as well as Eqs. (2) and (3), we get

$$\begin{aligned} 2 \sum_{P_i \in H} \phi_t^{M_{i-1}^*}(P_i) &\stackrel{(3)}{\geq} \sum_{P_i \in H \cup L} \phi_t^{M_{i-1}^*}(P_i) + \frac{1-t^2}{2} \cdot \text{OPT}_{j-1} \\ &\stackrel{(2)}{\geq} \frac{t-1}{2} \sum_{P_i \in H \cup L} c(P_i) + (1-t^2) \cdot \text{OPT}_{j-1} \\ &\geq \frac{t-1}{2} \sum_{P_i \in L} c(P_i) + (1-t^2) \cdot \text{OPT}_{j-1}. \end{aligned}$$

Due to the definition of t -net cost and the triangle equality, respectively, we have $t \cdot c(e_i) \geq \phi_t^{M_{i-1}^*}(P_i)$ and $c(P_i) \geq c(e_i)$. After dividing by $\frac{t-1}{2}$, this implies the lemma statement

$$\frac{4t}{t-1} \sum_{P_i \in H} c(e_i) \geq \sum_{P_i \in L} c(e_i) - 2(t+1) \cdot \text{OPT}_{j-1}.$$

□

Consider a level- k block B and denote by $\mathcal{B}$ the instance that consists of the requests and servers that are the endpoints of edges in M' belonging to B . Further denote by M'_B the set of these edges. We would like to bound the cost of M'_B for several disjoint blocks simultaneously using the previous lemma.

To be able to do this, we first need to argue, that it is okay to assume, for the sake of cost analysis, that the requests arrive in a certain order. In particular, we argue that as long as the relative order of requests with overlapping aggregate search intervals is preserved, the structure and cost of both the offline and the online matching somewhat remains unchanged. Specifically, when a request arrives, in any of such arrival orders satisfying the above, the offline and online matching are identical on the portion of the line corresponding to the requests aggregate search interval. In particular, when the n -th request arrives, the final matchings are identical.

Lemma 12 *Let $\pi \in S_n$ be a permutation such that $i < j$ and $I_i \subseteq I_j$ imply $\pi(i) < \pi(j)$. Changing the arrival order of requests to $r_{\pi(1)}, \dots, r_{\pi(n)}$ does not change the final matchings M' and M^* . In particular, after the change,*

- (i) *if r_i and r_j both belonged to block B , then so do $r_{\pi(i)}$ and $r_{\pi(j)}$ now, and*
- (ii) *the cost of M'_B remains unchanged compared to the original arrival order.*

Proof By definition of search intervals, only requests and servers inside the closure of $\bar{I}_i$ are considered by the t -net cost algorithm, and are thus the only ones possibly affected

by augmentation, when request r_i arrives. Therefore, we can assume by induction, that right before the arrival of r_i , the online and offline matching are identical within $\bar{I}_i$ for any permutation π that satisfies the above requirements. The t -net cost algorithm then uses the same path P_i for augmentation, irrespective of π , implying the lemma statement. $\square$

We are now ready to bound the cost of M' for a set of disjoint blocks.

Lemma 13 *For disjoint blocks $B_1, B_2, \dots, B_h$, and with $t = 3$, we have*

$$\sum_{i=1}^h c(M'_{\mathcal{B}_i}) \leq 5600 \cdot \text{OPT}.$$

Proof As we consider a set of disjoint blocks, we may use Lemma 12 and assume that the requests that belong to the blocks $B_1, B_2, \dots, B_h$ arrive consecutively, and that no requests on other portions of the line have arrived yet. Denote by M_L and M_H the set of long and short edges of $M'_{\mathcal{B}_1} \cup M'_{\mathcal{B}_2} \cup \dots \cup M'_{\mathcal{B}_h}$, respectively, and by $\text{OPT}_{\mathcal{B}_i}$ the cost of an optimal matching on $\mathcal{B}_i$. Together, Lemmas 3 and 13 from [29] imply

$$c(M_H) \leq 99 \cdot \sum_{i=1}^h \text{OPT}_{\mathcal{B}_i} + \frac{2}{47} \cdot c(M_L).$$

We then use Lemma 12 from [29] to obtain

$$c(M_H) \leq 594 \cdot \text{OPT} + \frac{2}{47} \cdot c(M_L).$$

Note that the lemma is only stated for a set of level- k blocks, but the proof carries over verbatim for the case of arbitrary disjoint blocks. Bounding $c(M_L)$ with Lemma 11, we get

$$c(M_H) \leq 594 \cdot \text{OPT} + \frac{2}{47} (6 \cdot c(M_H) + 8 \cdot \text{OPT}),$$

which in turn implies $c(M_H) \leq 798.2 \cdot \text{OPT}$. Using one last time Lemma 11, we conclude

$$\sum_{i=1}^h c(M'_{\mathcal{B}_i}) = c(M_H) + c(M_L) \leq 7 \cdot c(M_H) + 8 \cdot \text{OPT} \leq 5600 \cdot \text{OPT}.$$

$\square$

Proof of Lemma 8 To bound the cost of all active edges, recall that active edges are precisely those edges that are contained in a top block, and further, that at any given time any two top blocks are disjoint. Thus, we may apply Lemma 13 to the top blocks of M . This directly implies the statement of Lemma 8, concluding the competitive analysis of Algorithm 2. $\square$

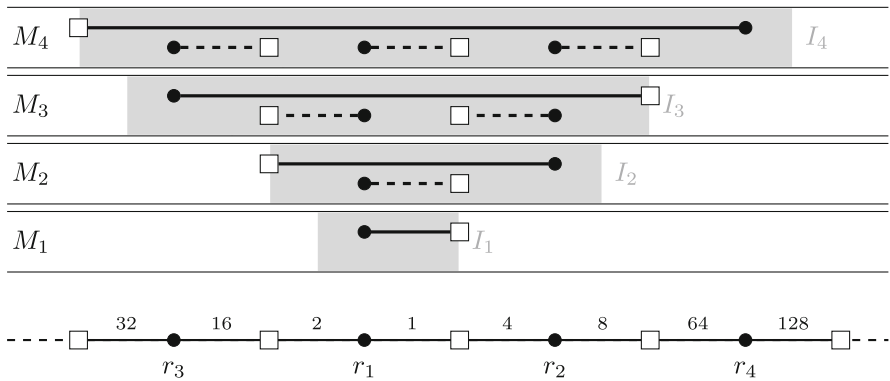


Fig. 4 An alternating instance with exponentially increasing connection costs. Edge costs are indicated above the corresponding portion of the line (drawing not to scale). Active edges are drawn solid, inactive edges dashed. Note that the aggregate search intervals (gray) grow exponentially in size. Therefore, a (top) block consists of a single aggregate search interval only and every arrival of a request triggers a recourse step, reassigning all existing inactive requests. This underlines the need for a freezing scheme

And the Recourse Budget? With the majority of requests being matched as in the offline matching M^* , it should come as no surprise that Algorithm 1 performs poorly in terms of recourse budget. To see this, consider the example in Fig. 4 showing an alternating instance with exponentially increasing edge costs. At the arrival of any request r_i with $i > 2$, all requests r_j with $j < i$ are reassigned. Thus, the recourse budget is linear and Algorithm 1 does not satisfy the requirements of Theorem 1. However, our efforts on Algorithm 1 are not in vain, since its shortcomings in terms of recourse budget can be remedied with a simple extension described in the following subsection.

3.2 Adding a Freezing Scheme to Obtain Constant Recourse

Consider again Fig. 4. The exponentially increasing edge costs that thwarted our success with Algorithm 1, turn out to also be the key to salvaging it. Intuitively, if edge costs do not increase too much, then a top block can hold more requests, so there will not be as many requests of different levels. As this translates directly into recourse actions, everything is already fine with Algorithm 1. If on the other hand edge costs do increase a lot, say exponentially as in Fig. 4, then at some point, the cost-contribution of small edges becomes negligible. Spending recourse actions on such edges would be absurd, so we instead choose to keep them as they are, we *freeze* them. We would still like to match other inactive requests that are not frozen with respect to M^* . However, since a frozen edge may block a server that, in M^* , is assigned to a different request r , we are forced to make a compromise and match r to a different, farther-away server via a *detour* taken along the frozen edge.

The Detour Matching We describe a way of combining two matchings that initially seem incompatible, e.g., because some request is matched to different servers in the two matchings. Let $\tilde{M}, M$ be matchings such that $\tilde{M} \oplus M$ consists of alternating paths

that are augmenting with respect to M . Such a path starts in a request and ends in a server, both of which are unmatched in M . We call the matching that consists of precisely these request-server pairs the *detour matching* of $\tilde{M}$ with respect to M . When clear from context, we simply refer to this matching as $\tilde{M} \oplus M$. For an illustration, see Fig. 5. For an example, consider the t -net cost algorithm. The symmetric difference of M_{i-1}^* and M_i^* consists of one path that is augmenting w.r.t. M_{i-1}^* , namely P_i . The detour matching of M_i^* w.r.t. M_{i-1}^* then consists of precisely the edge $e_i \in M_i'$, the *detour* that one takes when not augmenting along P_i but connecting the ends of the path directly. In metric spaces, it is easy to see via the triangle inequality, that the cost of the detour matching is bounded by the sum of costs of the two generating matchings.

Observation 14 *For matchings $\tilde{M}, M$ whose symmetric difference consists of augmenting paths with respect to M , the detour matching $\tilde{M} \oplus M$ satisfies $c(\tilde{M} \oplus M) \leq c(\tilde{M}) + c(M)$.*

A Simple Freezing Scheme Denote by $F \subseteq M'$ the set of frozen edges and consider the following simple freezing scheme for which we need to make the assumption that the precise values of OPT and n are known a priori. Freeze all edges of M' with cost $\frac{\text{OPT}}{n}$ or less and match inactive requests with respect to the detour matching $M^* \oplus F$. Since there are at most n edges in M' , the total cost of frozen edges is bounded by OPT , so by Observation 14, the cost of inactive edges is at most $c(F) + c(M^* \oplus F) \leq (t+2) \cdot \text{OPT}$. The cost of active edges remains in $O(\text{OPT})$ since nothing changed from Sect. 3.1. One can further show, that due to this freezing scheme, any request participates in only $O(\log n)$ recourse steps. The proof follows the intuition described above: If a request r would take part in $o(\log n)$ recourse steps, then it is contained in $o(\log n)$ intervals of increasing level, so edge costs are increasing exponentially and r must in fact already be frozen.

We would like to follow this general strategy as well. However, in the online model, we do not know OPT or n and, thus, need a dynamic freezing scheme, targeting for instance $\frac{\text{OPT}_i}{i}$ as a threshold for freezing. A typical guess-and-double approach may work concerning the costs. Yet, care has to be taken as $\frac{\text{OPT}_i}{i}$ is not monotone. A major obstacle appears to be the bounding of the recourse budget. The details of the algorithm and dynamic freezing scheme as well as their analysis are given in the remainder of this subsection.

3.2.1 Algorithm Description

Since the primary distinction from Algorithm 1 lies in the introduction of a freezing scheme, we begin by describing this addition. At time i , we say an inactive edge $e_j \in M'$ is frozen, if its cost is below $\frac{\text{OPT}_i}{i^2}$, and denote by F_i the set of all such edges. Furthermore, if for a previously frozen e_j we have that $c(e_j) > \frac{\text{OPT}_i}{i^2}$, then we *unfreeze* the edge and delete it from F_i . Note that we use the edge e_j of the online matching to determine whether r_j is frozen, but r_j is possibly matched in M_i via a different edge.

While we generally aim to match frozen requests as in the online matching M' , this assignment is not implemented until the request takes part in a recourse step. We show that this prevents unnecessary recourse actions that could be caused by requests

repeatedly alternating between being frozen and unfrozen. Denote by $M_i^F \subseteq F_i$ the subset of frozen edges that is in M_i , i.e., the set of edges where the aforementioned assignment according to M' was implemented during a recourse step. When unfreezing a request, however, there is no delay and the corresponding change must be reflected in the matching immediately, since the cost contribution of the frozen edge is now too high.

With the addition of the freezing scheme, our algorithm, summarized as Algorithm 2, now partitions the requests into three groups: Active requests matched according to the online matching M' , inactive, frozen requests in M^F , also matched according to M' , and inactive requests that are not in M^F , matched according to the detour matching $M^* \oplus M^F$. Similarly, the algorithm itself is now divided into three steps, that are all executed when a new request arrives:

Algorithm 2 : Matching Recourse with Freezing Scheme

Upon arrival of the i -th request r_i : ▷ r_i matched in M' via $e_i = (r_i, s_i)$

Matching r_i

1: match r_i to s_i as in M' ▷ label r_i active

Freezing/Unfreezing

2: determine the set F_i of frozen edges

3: **for** inactive requests r_j that become unfrozen **do**

4: remove e_j from M_i^F and repair assignments on corresponding path of $M_{h(i,j)}^* \oplus M_i^F$

Recourse step

5: **if** there is an active $r_j \in I_i$ with $\ell(I_j) < \ell(I_i)$ **then** ▷ I_i creates new top block

6: **for** $r_j \in I_i$ recently frozen **do** ▷ label $r_j \in I_i \setminus \{r_i\}$ inactive

7: add $e_j = (r_j, s_j)$ to M_i^F and reassign r_j to s_j

8: **for** unfrozen $r_j \in I_i$ with $j \neq i$ **do**

9: reassign r_j according to $M_{i-1}^* \oplus M_i^F$

Matching r_i Upon arrival of request r_i , label it active and match it according to the online matching M' , that is, via the edge $e_i = (r_i, s_i)$.

Freezing/Unfreezing Update the set F_i of frozen edges according to the freezing rules. If a previously frozen e_j is unfrozen, i.e., $c(e_j) > \frac{\text{OPT}_i}{i}$, then unfreeze it and delete it from F_i and M_i^F . Reassign it with respect to $M_{h(i,j)}^* \oplus M_i^F$, with $h(i, j) = \max\{k \leq i \mid r_j \in I_k \text{ and } I_k \text{ is not in a top block}\}$. For an illustration of $h(i, j)$, see Fig. 3, where $h(i, \ell) = j$. Reassignments due to unfreezing are efficiently implemented as follows.

Consider an edge $e = (s, r)$ right before it is unfrozen. If e is already matched according to the offline matching, then there is nothing to do. Otherwise, it must be part of an alternating path or cycle in $M^* \oplus M^F$. In the latter case, again, no recourse action needs to be taken as the removal of e from M^F results in a path from r to s . In the detour matching, we want to connect the ends of this path, which is already accomplished by the edge $e = (s, r)$. Consider the case that $e \in P$ for some alternating path P in $M^* \oplus M^F$ that starts in a request r' and ends in a server s' . Unfreezing e and matching according to $M^* \oplus M^F$ decomposes P into the r' – s -path P_1 and the r – s' -path P_2 . In Algorithm 1, we implement these changes via two recourse actions: we reassign r to s' and r' to s ; see Fig. 5.

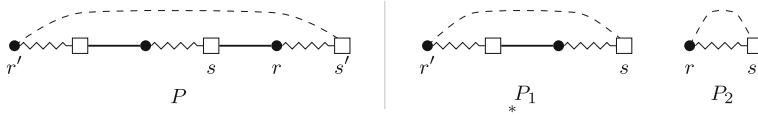


Fig. 5 Illustration of Step 4 in Algorithm 2. Edges of M^* are drawn in a zigzag pattern, edges of M^F solid and edges of the detour matching $M^* \oplus M^F$ dashed. After unfreezing r , the removal of (r, s) splits the path P in $M^* \oplus M^F$ in two augmenting paths P_1 and P_2

Recourse step If there is no $j < i$ such that $I_j \subseteq I_i$ and $\ell(I_j) = \ell(I_i)$, the arrival of r_i produces a new top block and triggers a recourse step. In this case, we assign requests in I_i that were recently frozen according to M' and add the corresponding edge to M_i^F . Next, we reassign all other requests (unfrozen, inactive) that lie in I_i according to $M_{i-1}^* \oplus M_i^F$, the detour matching.

3.3 Algorithm Analysis

We start again by verifying that Algorithm 2 produces a matching after each iteration. As before, r_i can be matched to s_i in Step 1. Similarly, active edges do not interfere in the unfreezing procedure (Step 4) due to the definition of $h(i, j)$. It remains to argue that we can indeed construct the detour matching as described. To this end, we consider the symmetric difference $M^* \oplus M^F$ and show that, for an unfrozen request r , there is an augmenting path P from r to a server s' not covered by M^F . To see this, note that in $M^* \oplus M^F$ the unfrozen r is incident only to an edge $(r, s) \in M^*$, so it cannot be contained in a cycle. Additionally, the path starting with (r, s) cannot end at a request, since the remaining requests on the path are covered by an edge of M^F and M^* each. Thus, it must end at a server s' that is not matched in M^F , as required.

Observation 15 *At the end of each iteration, Algorithm 2 produces a feasible matching M_i .*

When bounding the competitive ratio, we may use the results from Sect. 3.1. By Lemma 8, the cost of active edges is in $O(\text{OPT})$. Using Observation 14 and the definition of the freezing scheme, we bound the cost of inactive edges by $c(M^F) + c(M^* \oplus M^F) \leq (t + 2) \cdot \text{OPT}$. Together, this implies a constant competitive ratio.

Corollary 16 *Algorithm 2 has a competitive ratio of $O(1)$.*

Bounding the recourse Due to delaying reassignments until an appropriate recourse step, freezing a request r does not cause any immediate recourse actions. Unfreezing r , on the other hand, may cause reassignments (Step 4), namely of r and possibly one additional request, see Fig. 5. We charge these two recourse actions to r , and in particular, to the last recourse step it participated in. However, recourse due to unfreezing r happens at most once between two consecutive recourse steps of r . This implies, that a request is charged at most three times per recourse step it participates in, once for the recourse step itself and possible two times more for a subsequent unfreezing.

Observation 17 *A request is charged at most three times the number of recourse steps it is involved in.*

For a request that gets frozen at time i and unfrozen at time $j > i$, we have $c(e) \leq \frac{\text{OPT}_i}{i^2}$ and $c(e) > \frac{\text{OPT}_j}{j}$ for the corresponding edge $e \in M'$. Since $\text{OPT}_i \leq \text{OPT}_j$, this implies the following.

Observation 18 *A request frozen at time i stays frozen at least until time i^2 .*

On the other hand, the number of recourse steps in which a continuously unfrozen request takes part in can be bounded from above.

Lemma 19 *If a request $r \in I_i \subseteq I_j$ is not frozen from time i to time j , then it holds that $\ell(I_j) - \ell(I_i) = O(\log j)$. In particular, between time i and j , there are $O(\log j)$ recourse steps in which r can participate.*

Proof Consider the search interval $\bar{I}_j = (s_j^L, s_j^R)$. By definition of search intervals, there exist augmenting paths P_j^L, P_j^R connecting r_j to s_j^L and s_j^R with the same t -net cost as P_j , the path used by the t -net cost algorithm. We bound the length of $P_j \in \{P_j^L, P_j^R\}$ by

$$c(P_j) = c(P_j \cap M_{j-1}^*) + c(P_j \cap M_j^*) \leq 2t \cdot \text{OPT}_j. \quad (4)$$

Without loss of generality, assume $P_j = P_j^L$. Interpreting the point s_j^R as a virtual server and assuming that it is contained in S , we could augment M_{j-1}^* also along P_j^R yielding a different matching $\tilde{M}_j^*$. By definition of P_j^R and Eq. (1), we have

$$\begin{aligned} t \cdot c(P_j \cap M_j^*) - c(P_j \cap M_{j-1}^*) &= \phi_t^{M_{j-1}^*}(P_j) = \phi_t^{M_{j-1}^*}(P_j^R) \\ &= t \cdot c(P_j^R \cap \tilde{M}_j^*) - c(P_j^R \cap M_{j-1}^*). \end{aligned}$$

Thus,

$$\begin{aligned} c(P_j^R) &= c(P_j^R \cap M_{j-1}^*) + c(P_j^R \cap \tilde{M}_j^*) \\ &\leq c(P_j^R \cap M_{j-1}^*) + c(P_j \cap M_j^*) + \frac{1}{t}c(P_j^R \cap M_{j-1}^*) \\ &\leq 3t \cdot \text{OPT}_j. \end{aligned}$$

From Eq. (4), we get $|\bar{I}_j| \leq c(P_j^L) + c(P_j^R) \leq 5t \cdot \text{OPT}_j$, which implies $|I_j| \leq \sum_{k \leq j} |\bar{I}_k| \leq 5t \cdot j \cdot \text{OPT}_j$. On the other hand, the cost of $|I_i|$ can be bounded from below by $c(e)$, where e is the edge in M' incident to r , as both endpoints of e are contained in the interval. We then obtain

$$\frac{|I_j|}{|I_i|} \leq \frac{5t \cdot j \cdot \text{OPT}_j}{c(e)} < 5t \cdot j^3.$$

The last inequality follows from the assumption that r is not frozen at time j , implying $c(e) > \frac{\text{OPT}_j}{j^2}$. Recall that $|I_i| < (1 + \varepsilon)^{\ell(I_i)}$ and $(1 + \varepsilon)^{\ell(I_j)-1} \leq |I_j|$. We

conclude

$$\ell(I_j) - \ell(I_i) \leq 1 + \log_{(1+\varepsilon)}(5t \cdot j^3) = 1 + \log_{(1+\varepsilon)}(5t) + \frac{3 \log j}{\log(1+\varepsilon)} \leq c \cdot \log j, \quad (5)$$

for some constant c . Regarding the second claim, recall that r participates in a recourse step when an interval containing r opens a new top block (i.e. a new level) while r is not (freshly) frozen. Between time i and time j , this can only happen for intervals I_h of distinct levels for which $I_i \subseteq I_h \subseteq I_j$. The claim then directly follows Eq. (5). $\square$

Lemma 19 and Observation 18 bound the total number of recourse actions taken by Algorithm 2.

Lemma 20 *Algorithm 2 uses a recourse budget of at most $O(\log n)$.*

Proof Consider a request r . By Observation 17, it suffices to bound the number of recourse steps that r is involved in. Let $[i_h^U, i_h^F]$, for $h = 0, 1, \dots, k$, be maximal intervals of consecutive time points during which r is not frozen, i.e., r is not frozen only at the time points i with $i \in [i_h^U, i_h^F]$ for some h . We use induction on k to show that r participates in at most $2c \cdot \log(i_k^F)$ recourse steps, where c is the constant from Eq. (5). The base case, $k = 0$, follows directly from Lemma 19. For $k \geq 1$, we have $(i_{k-1}^F)^2 \leq i_k^U \leq i_k^F$ due to Observation 18. By induction hypothesis, the number of reassignments that involve r in the first $k - 1$ time intervals is at most

$$2c \cdot \log(i_{k-1}^F) \leq 2c \cdot \log\left(\sqrt{i_k^F}\right) = c \cdot \log(i_k^F).$$

For the last time interval, we have at most $c \cdot \log(i_k^F)$ many such recourse steps by Lemma 19. Since $i_k^F \leq n$, this concludes the proof. $\square$

Proof of Theorem 1 Corollary 16 and Lemma 20, together, directly imply the validity of Theorem 1. This concludes the analysis of Algorithm 2. $\square$

3.4 A Scalable Algorithm

With Algorithm 2, we have given an algorithm that has a constant competitive ratio at the price of a recourse budget that is bounded by $O(\log n)$. While the focus was on the former, i.e., on achieving a constant competitive ratio, it may be of practical interest to more flexibly balance the cost performance of the algorithm with its recourse budget instead, as described in Theorem 2. Indeed, all the building blocks needed for the proof of this theorem have already been established in the previous subsections. As we now show, a slight alteration to Algorithm 2 already yields the tradeoff between cost and recourse we desire.

Theorem 2 *For any function $f : \mathbb{N} \rightarrow \mathbb{N}$ with $1 \leq f(n) \leq \log n$, there is an algorithm for online metric matching on the line that is $O(f(n))$ -competitive and has an amortized recourse budget of $O\left(\frac{\log n}{f(n)}\right)$.*

Proof To obtain an algorithm that is $O(f(n))$ -competitive and uses a recourse budget of $O(\frac{\log n}{f(n)})$, we alter Algorithm 2 as follows. Intuitively, and in the terms of the high-level overview from the beginning of this section, we want the input buffers (top blocks) to have a higher capacity (more active requests), leading to a larger cost due to the contribution of edges in M' but lower recourse due to less frequent updates. We achieve this by “vertically” (consider the visualization in Fig. 3) enlarging the area of top blocks and allowing the requests of several non-disjoint blocks to be active. However, intersecting active blocks may not differ in their level by more than $f(n)$, which specifies the “height” of the input buffer.

Specifically, all we do is change Step 5 of Algorithm 2 as follows.

5: **if** there is an active $r_j \in I_i$ such that $\ell(I_j) \leq \ell(I_i) - f(n)$ **then**

It is easy to see, that the cost of active edges is then in $O(f(n) \cdot \text{OPT})$, since the set of blocks with active requests can be partitioned into at most $f(n)$ sets of disjoint blocks to which our original analysis can be applied separately.

Similarly, when considering the recourse budget, note that the first part of the statement of Lemma 19 remains true, but now implies that an unfrozen request can participate in at most $O(\frac{\log j}{f(n)})$ recourse steps. Lemma 20 can easily be adapted to account for the additional factor of $\frac{1}{f(n)}$ which concludes the proof of the theorem. $\square$

To see that our analysis is asymptotically optimal, we consider the family of lower bound instances (for the online setting) described in [2] and give a brief sketch of its construction. An instance of the family consists of $n = 2^h - 1$ requests and $n + 1$ servers that alternate on the line. That is, requests are placed at points $2i - 1$ for $i \in [n]$ and servers at points $2i - 2$ for $i \in [n + 1]$, where $[n]$ denotes the set $\{1, 2, \dots, n\}$. The requests appear in h batches, each of which contains every second request of those that have not appeared yet: Initially, requests $4i - 3$ for $i \in [2^{h-2}]$, then requests $8i - 5$ for $i \in [2^{h-3}]$, and so on. Restricting the class of considered algorithms by some assumptions, which are satisfied for all known deterministic online algorithms to date, one can show the following properties. Any such algorithm matches a requests of batch i at minimum cost to either the nearest free server to the left or to the right. When partitioning requests of a batch into pairs of consecutive requests, the requests of a pair match towards each other, i.e., to the two free servers between the two requests. Between those two servers, in turn, always lies a single request which is contained in the next batch. Thus, the cost of an edge in batch i is precisely 2^{i-1} , which shows the lower bound. For details, see [2].

Structurally, the exponentially increasing edge costs mean that every arriving request creates a new top block, similarly as in Fig. 4. Additionally, the structure of aggregate search intervals resembles a full binary tree. Specifically, there are $\log n$ levels and all $\frac{n+1}{2}$ requests on the lowest level will participate in $\frac{\log n}{f(n)}$ recourse steps due to the arrival of their higher level ancestors. At the same time, no request is frozen, as $\text{OPT}_i = i$ and the minimum cost of an edge is 1. Therefore, we have a lower bound of $\Omega(\frac{\log n}{f(n)})$ for the recourse budget of our algorithm.

4 A Near-Optimal Algorithm on Alternating Instances

In this section, we consider alternating instances, i.e., instances on which requests and servers alternate on the line. We assume without loss of generality that $|S| = |R|$. For such instances, an optimum (offline) matching assigns all requests either to the server directly to their left or all requests to the server on their right. Denote these matchings by M^L and M^R , respectively, and call their edges *minimal*.

We describe a $(1 + \varepsilon)$ -competitive algorithm for alternating instances that reassigns each request at most a constant number of times. In addition to its output M , it maintains M^* , the matching produced by the offline t -net cost algorithm, and a set of frozen edges M^F . In contrast to Algorithm 2, we are not only able to disregard M' , but furthermore we can employ a much simpler freezing scheme. Specifically, a request is frozen when it is reassigned for the k -th time, for some k only depending on ε . The request remains matched to its current server perpetually and the corresponding edge is added to M^F . Non-frozen requests are matched according to the detour matching $M^* \oplus M^F$ as described in Sect. 3. By design, the recourse budget per request is constant, only the competitive analysis remains. The algorithm is summarized as Algorithm 3.

Observe that when considered as line segments, the augmenting paths P_i used by the t -net cost algorithm have a laminar structure on alternating instances, since after time i , there are no free requests or servers in the portion of the line corresponding to P_i . This allows us to view augmenting paths as nodes of a forest, the *path forest*, where P_i is a child of the minimal augmenting path that properly contains it, or a root if no such path exists (see Fig. 6). The *depth* of a path P_i denotes its distance to the root of its tree and determines the number of reassignments of the corresponding request r_i in M^* .

The intuition why such an extremely simple freezing scheme as utilized in Algorithm 3 works on alternating instances, relies on the fact that, here, we can draw a direct connection between the length of a path P_i and its depth in the path forest. Specifically, we will show in Lemma 23, that the lengths of augmenting paths grow exponentially when moving from a leaf of the path forest to the corresponding root. Since ancestors of a request correspond to reassignments in M^* , we can charge the cost of frozen edges of a certain depth as well as possible detours to higher level ancestors. The per-request recourse budget is thus directly connected to the competitive ratio

Algorithm 3 : Algorithm for alternating instances with per-request recourse budget $k \in \mathbb{N}$

initialize recourse counter $h_r \leftarrow 0$ for all $r \in R$, $M^F \leftarrow \emptyset$

Upon arrival of the i -th request r_i :

- 1: update M_{i-1}^* by augmenting along P_i , and set $h_r \leftarrow h_r + 1$, for all $r \in P_i$
 - 2: **for all** $r \in P_i$ with $h_r > k$ **do** $\triangleright r$ frozen
 - 3: $M^F \leftarrow M^F \cup \{(r, s)\}$, where s is the server currently matched to r
 - 4: **for all** $r \in P_i$ with $h_r \leq k$ **do** $\triangleright r$ not frozen
 - 5: reassign r with respect to $M_i^* \oplus M^F$
-

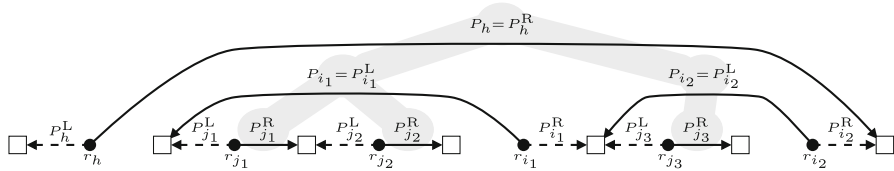


Fig. 6 Illustration of a path tree in an alternating instance. Paths not chosen for augmentation are dashed. Servers are depicted as squares and requests as filled circles

since a larger recourse budget means charging to a higher ancestor with exponentially larger cost.

Notation We use a similar interval structure as before and keep the notation. The definition of the intervals is slightly changed as follows. Consider intervals $I_i = [s_i^L, s_i^R]$, where $s_i^L, s_i^R \in S$ are the closest free servers on the line to the left and right of r_i respectively at the time of its arrival. For this to be well-defined, we possibly need place an additional server at ∞ or $-\infty$. Denote by P_i^L, P_i^R the alternating paths connecting r to s_i^L and s_i^R , respectively, that have shortest t -net cost. Recall, that for the t -net cost algorithm there are no free servers inside the search interval. Thus, $\bar{I}_i \subseteq I_i$ and the following observation holds.

Observation 21 We have $P_i \in \{P_i^L, P_i^R\}$.

The following lemma establishes some properties of the offline t -net cost algorithm on alternating instances.

Lemma 22 (i) Paths P_i^L, P_i^R and matching M_i^* only use minimal edges.

(ii) If $P_i = P_i^X$, for $X \in \{L, R\}$, on I_i , locally, we have that $M_i^* = M^X$.

Specifically, this implies $\phi_i^{M_{i-1}^*}(P_i) = \phi_i^{M^Y}(P_i)$ for $X \neq Y \in \{L, R\}$.

(iii) If P_j is a child of P_i , then $P_i = P_i^L$ if and only if $P_j = P_j^R$.

In particular, we have $I_j \subseteq P_i$ and $I_j \cap M_j^* \cap M_i^* = \emptyset$.

Proof Due to symmetry, we assume without loss of generality that $P_i = P_i^R$.

(i): We use induction on i . The base case, $i = 1$, is easy, so let $i \geq 2$. Suppose, for sake of contradiction, that P_i contains a non-minimal edge. Given a choice, consider the non-minimal edge e which is closest to s_i on P_i . Moving from r_i to s_i along P_i , edges leading from a server to a request are in M_{i-1}^* and by induction hypothesis minimal. Thus, $e = (r, s) \in M_i^*$ leads from r to s .

Consider the server s' directly to the right of r . Since e is not minimal, we have $r < s' < r' < s$, with r' being the request right of s' . We first consider the case that s' is free. Observation 21 implies that $s' = s_i$. We show that altering P_i to go from r directly to s' would yield a path of strictly lower t -net cost. To this end, first note that the remaining edges on P_i after s are minimal since e is closest to s_i . Thus, the t -net cost of the subpath of P_i starting at r is at least $t \cdot c(r, s) - c(s', s) = t \cdot c(r, s') + (t-1) \cdot c(s', s) > t \cdot c(r, s')$, with the last term being the t -net cost of connecting r directly to s' .

If s' is matched on the other hand, then it must be matched to r' by induction hypothesis. Therefore, replacing e by $(r, s'), (s', r'), (r', s)$ reduces the t -net cost by $(t+1) \cdot c(s', r')$, contradicting the minimality of P_i .

(ii): By Statement (i), P_i^L and P_i^R only consist of minimal edges. Moving along P_i from left to right, edges from server to request are in M_{i-1}^* and edges from request to server in M_i^* . This is simply due to the fact that P_i is a path that augments M_{i-1}^* to form M_i^* , and it starts at a request and ends at a server. We therefore have $M_{i-1}^* \cap P_i^R \subseteq M^L$. With a symmetrical argument, we obtain $P_i^L \cap M_{i-1}^* \subseteq M^R$. After augmenting M_{i-1}^* along P_i , the edges point in the opposite direction and the claim follows.

(iii): By (ii), we know that $M_j^* \cap I_j = M^X$, say $X = R$, so edges are of the form (r, s) with $r < s$. Due to Statement (i), only augmenting paths leading from right to left can augment along such edges. If this happens, all of I_j is traversed as there is no free server in its interior. As parent of P_j , P_i is the first path to properly contain P_j and thus $P_i = P_i^L$, and in particular, $I_j \subseteq P_i$. The equation $I_j \cap M_j^* \cap M_i^* = \emptyset$ then follows directly from (ii). $\square$

Note that due to Lemma 22 (i), the sum of edge costs of an augmenting path is exactly equal to the length of the corresponding line segment. In particular, we have $c(P_i^L) + c(P_i^R) = |I_i|$. For simplicity, we abuse notation and use I_i to denote both, the interval on the line, and the set of edges $P_i^L \cup P_i^R$.

Lemma 22 can further be used to show that when considering the sum of lengths of augmenting paths of a certain depth in the path forest, then this number increases exponentially as the depth decreases, i.e., towards a root. Fix an augmenting path P_h and consider the induced subtree of augmenting paths with root P_h . Denote by $\mathcal{H}_k$ the set of indices of paths at depth k with P_h being at depth 0.

Lemma 23 Consider a path P_h and its grandchildren, i.e., P_j with $j \in \mathcal{H}_2$. Then

$$c(P_h) \geq \left(2 - \frac{1}{t}\right) \cdot \sum_{j \in \mathcal{H}_2} c(P_j).$$

Proof Denote by P_i , $i \in \mathcal{I} = \mathcal{H}_1$, the children of P_h in the path-forest and by $\mathcal{J}_i \subseteq \mathcal{J} = \mathcal{H}_2$ the sets of indices of their respective children. Without loss of generality, assume $P_h = P_h^R$. Lemma 22 (iii), implies $P_i = P_i^L$, and $P_j = P_j^R$, for $i \in \mathcal{I}$, $j \in \mathcal{J}$; see Fig. 6. With again Lemma 22 (iii), and using the fact that $\phi_t^{M_{i-1}^*}(P_i^R) \geq \phi_t^{M_{i-1}^*}(P_i^L)$, by definition of the t -net cost algorithm, we get

$$\begin{aligned} t \cdot c(P_h) &\geq t \cdot \sum_{i \in \mathcal{I}} (c(P_i^L) + c(P_i^R)) \\ &\geq t \cdot \sum_{i \in \mathcal{I}} c(P_i^L) + \sum_{i \in \mathcal{I}} \phi_t^{M_{i-1}^*}(P_i^R) \\ &\geq t \cdot \sum_{j \in \mathcal{J}} c(P_j^R) + t \cdot \sum_{i \in \mathcal{I}} c(P_i^L \setminus (\cup_{j \in \mathcal{J}_i} P_j^R)) + \sum_{i \in \mathcal{I}} \phi_t^{M_{i-1}^*}(P_i^L). \end{aligned} \quad (6)$$

Using Lemma 22 (ii), we get

$$\phi_t^{M_{i-1}^*}(P_i^L) \stackrel{(ii)}{=} \phi_t^{M^R}(P_i^L)$$

$$\begin{aligned}
 &= \sum_{j \in \mathcal{J}_i} \left(\phi_t^{M^R}(P_j^L) + \phi_t^{M^R}(P_j^R) \right) + \phi_t^{M^R}(P_i \setminus (\cup_{j \in \mathcal{J}_i} I_j)) \\
 &\stackrel{(ii)}{\geq} \sum_{j \in \mathcal{J}_i} \left(\phi_t^{M^{j-1}}(P_j^L) + \phi_t^{M^R}(P_j^R) \right) - t \cdot c(P_i \setminus (\cup_{j \in \mathcal{J}_i} I_j)).
 \end{aligned}$$

Since $\phi_t^{M^{j-1}}(P_j^L) \geq \phi_t^{M^{j-1}}(P_j^R) = \phi_t^{M^L}(P_j^R)$, the above together with Inequality (6) implies

$$t \cdot c(P_h) \geq \sum_{j \in \mathcal{J}} \left(t \cdot c(P_j^R) + \phi_t^{M^L}(P_j^R) + \phi_t^{M^R}(P_j^R) \right) \geq (2t - 1) \cdot \sum_{j \in \mathcal{J}} c(P_j^R),$$

where last inequality follows from the observation that, due to Eq. (1), it holds that $\phi_t^{M^L}(P) + \phi_t^{M^R}(P) = (t - 1) \cdot c(P)$. $\square$

We are now equipped to prove Theorem 3.

Theorem 3 *For alternating instances of online metric matching on the line, there is a $(1 + \varepsilon)$ -competitive algorithm that reassigns each request $O(\frac{1}{\varepsilon^2})$ times.*

Proof Fix an augmenting path P_h and consider its descendants P_j at depth $2k + 2$ in the path tree rooted at P_h , i.e., $j \in \mathcal{H}_{2k+2}$. The intervals I_j are contained in paths $P_{j'}$, $j' \in \mathcal{H}_{2k+1}$, by Lemma 22 (iii). Lemma 23 implies

$$\sum_{j \in \mathcal{H}_{2k+2}} |I_j| \leq \sum_{j' \in \mathcal{H}_{2k+1}} c(P_{j'}) \leq \left(2 - \frac{1}{t}\right)^{-k} \cdot \sum_{i \in \mathcal{H}_1} c(P_i). \quad (7)$$

Raghvendra [28] shows that the t -net cost of augmenting paths is always non-negative. In particular, $\phi_t^{M_h^{*-1}}(P_h) = t \cdot c(P_h \cap M_h^*) - c(P_h \cap M_{h-1}^*) \geq 0$. Thus,

$$\sum_{i \in \mathcal{H}_1} c(P_i) \leq c(P_h) = c(P_h \cap M_h^*) + c(P_h \cap M_{h-1}^*) \leq (t + 1) \cdot c(P_h \cap M_h^*). \quad (8)$$

Similarly,

$$\sum_{i \in \mathcal{H}_1} c(P_i) \leq (t + 1) \cdot \sum_{i \in \mathcal{H}_1} c(P_i \cap M_i^*) \leq (t + 1) \cdot c(P_h \cap M_i^*). \quad (9)$$

On an interval I_i with $i \in \mathcal{H}_1$, locally, $M_h^* = M^L$ if and only if $M_i^* = M^R$ by Lemma 22 (ii). For $X = L$ and $X = R$, Eq. (7) together with Eqs. (8) and (9), respectively, implies

$$\sum_{j \in \mathcal{H}_{2k+2}} |I_j| \leq \left(2 - \frac{1}{t}\right)^{-k} \cdot (t + 1) \cdot c(P_h \cap M^X).$$

Rearranging, we obtain

$$\left[\frac{1}{t+1} \left(2 - \frac{1}{t} \right)^k - 1 \right] \cdot \sum_{j \in \mathcal{H}_{2k+2}} |I_j| \leq c \left(P_h \setminus (\cup_{j \in \mathcal{H}_{2k+2}} I_j) \cap M^X \right). \quad (10)$$

Denote by $\alpha(k, t)$ the term in square brackets. When a (minimal) edge (r, s) is frozen, the remaining instance on $S \setminus \{s\} \times R \setminus \{r\}$ is again alternating and at most one request will take a detour due to (r, s) being frozen. The additionally incurred cost when compared to M^* is bounded by $2 \cdot |I_r|$ and can, via Inequality (10), be charged to non-frozen parts of M^{OPT} (recall that $M^{\text{OPT}} \in \{M^L, M^R\}$). The set difference $P_h \setminus (\cup_{j \in \mathcal{H}_{2k+2}} I_j)$ in Inequality (10) ensures that the portion $P_h \cap M^{\text{OPT}}$ of M^{OPT} is charged at most $2k + 2$ times before P_h itself is frozen and the cost is charged to another portion of M^{OPT} . This leads to a competitive ratio of $(t + \frac{2k+2}{\alpha(k,t)})$.

We set $t = 1 + \varepsilon$ and show that this results in a competitive ratio of $1 + 2\varepsilon$ for an appropriate choice of k . This is equivalent to showing that

$$2k + 2 \leq \varepsilon \cdot \alpha(k, t) = \frac{\varepsilon}{2 + \varepsilon} \cdot \left(1 + \frac{\varepsilon}{1 + \varepsilon} \right)^k - \varepsilon.$$

Assuming $\varepsilon < 2$ and using $\left(1 + \frac{\varepsilon}{1 + \varepsilon} \right)^{\frac{4}{\varepsilon}} \geq e$ for $\varepsilon < 2$, this can be further simplified to

$$k \cdot \frac{10}{\varepsilon} \leq (e^{\frac{\varepsilon}{4}})^k.$$

Setting $k = 30 \cdot \varepsilon^{-2}$, this statement is true for all $\varepsilon < 2$. This concludes the proof of the theorem. $\square$

As a byproduct, we show, for this special class of instances, a result in the online setting without recourse. It relates the competitive ratio to the cost metric, i.e., the maximum difference in edge cost for connecting a request to a server and refines the best known competitive ratio of $O(\log n)$ by Raghvendra [29]. The proof of the following theorem additionally contains a proof for an upper bound of $O(\log n)$ for alternating instances that is much simpler than the general proof by Raghvendra [29].

Theorem 24 *The online t -net cost algorithm is $O(\min\{\log n, \log \Delta\})$ -competitive, with $\Delta = \max_{r, r' \in R, s, s' \in S} \frac{c(r, s)}{c(r', s')}$, for alternating instances of online metric matching on the line.*

Proof Consider an edge e_i of M' and assume $P_i = P_i^R$. By Lemma 22 (ii) and the definition of the t -net cost algorithm,

$$\begin{aligned} t \cdot c(P_i^L \cap M^L) - c(P_i^L \cap M^R) &= \phi_t^{M_{i-1}^*}(P_i^L) \\ &\geq \phi_t^{M_{i-1}^*}(P_i^R) = t \cdot c(P_i^R \cap M^R) - c(P_i^R \cap M^L), \end{aligned}$$

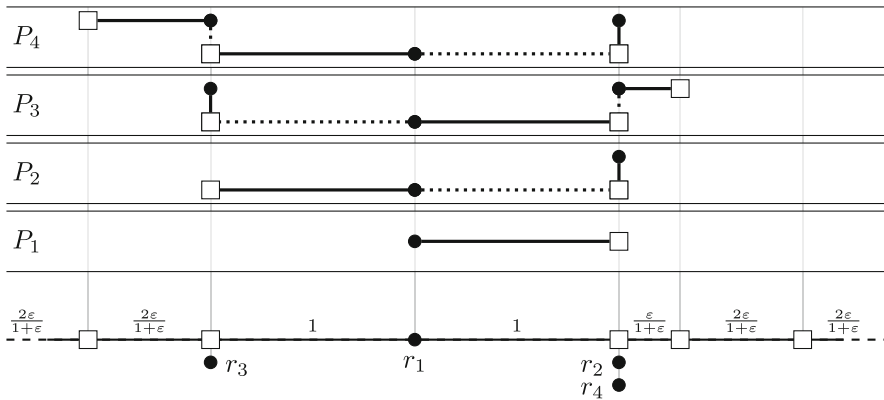


Fig. 7 A non-alternating instance, where the strategy from Sect. 4 fails. Edge costs are indicated above the corresponding portion of the line and the paths P_i are the minimum t -net-cost paths chosen by the offline matching M^* for augmentation ($t = 1 + \varepsilon$). In each iteration, request r_1 is reassigned. However, the cost of the optimum solution does not increase exponentially. More precisely, $\text{OPT}_i = 1 + \sum_{j=1}^{i-1} \frac{j \cdot \varepsilon}{1 + \varepsilon}$. It is therefore not possible to freeze request r_1 after a constant number of reassignments. Note, however, that the amortized recourse budget is constant, even without freezing

which implies

$$c(P_i^R \cap M^R) \leq \frac{1}{t} \cdot c(P_i^R \cap M^L) + c(P_i^L \cap M^L).$$

Therefore,

$$\begin{aligned} c(P_i) &= c(P_i^R \cap M^L) + c(P_i^R \cap M^R) \\ &\leq \left(1 + \frac{1}{t}\right) \cdot c(P_i^R \cap M^L) + c(P_i^L \cap M^L). \end{aligned} \quad (11)$$

As in Eq. (8), we use $\phi_t^{M_{i-1}^*}(P_i) = t \cdot c(P_i \cap M_i^*) - c(P_i \cap M_{i-1}^*) \geq 0$ to obtain

$$\begin{aligned} c(P_i) &= c(P_i \cap M_i^*) + c(P_i \cap M_{i-1}^*) \\ &\leq (t + 1) \cdot c(P_i \cap M_i^*) \\ &= (t + 1) \cdot c(P_i \cap M^R). \end{aligned} \quad (12)$$

Together, Inequalities (11) and (12) imply

$$c(e_i) = c(P_i) \leq c(I_i \cap M^{\text{OPT}}) \cdot \max\{t + 1, 1 + \frac{1}{t}\}.$$

Since intervals corresponding to the same depth in the path-forest are disjoint, we can bound the lengths of paths which are of identical depth by $\max\{t + 1, 1 + \frac{1}{t}\} \cdot \text{OPT}$. As there are at most $2 \cdot \log_{(2-1/t)} \Delta$ different depths in total due to Lemma 23, this bounds the competitive ratio by $O(\log \Delta)$.

Further, using the same approach, we can directly obtain the upper bound of $O(\log n)$ from Raghvendra [29]. Specifically, we bound the cost of the nodes of depth

$O(\log n)$ or lower in the path forest by $O(\log n) \cdot \max\{t + 1, 1 + \frac{1}{t}\} \cdot \text{OPT}$ as above. For nodes P_j of larger depth, we denote by P_i the root of the corresponding path tree and apply Lemma 23 repeatedly, $O(\log n)$ times, to bound their cost by

$$c(P_j) \leq O\left(\frac{1}{n}\right) \cdot c(P_i) \leq \frac{1}{n} \cdot O(\text{OPT}).$$

Noting that there are less than n such edges completes the proof. We close this section by showing that the strategy that we have proposed in Sect. 4 fails in the general case. Figure 7 demonstrates an example. $\square$

5 Conclusion

In this paper, we give non-trivial results for the online metric bipartite matching problem with recourse. The results were obtained simultaneously with and independently of Gupta et al. [15] who consider also more general metrics than the line. We confirm that an average recourse of $O(\log n)$ per request is sufficient to obtain an $O(1)$ -competitive matching on the line. It remains open if such a result can be obtained in a non-amortized setting, where the recourse is available only per iteration. Our algorithm is clearly designed for the amortized setting as it buffers online matching decisions and repairs them in batches.

Further, it remains open whether constant recourse per request suffices for maintaining an $O(1)$ -competitive matching on the line, as for the case of alternating requests. This may be very well possible as there is, currently, no lower bound that rules this out. While the strategy proposed in Sect. 4 fails in the general case, we nevertheless hope that the ideas provided in this paper still prove helpful for solving this open question.

Funding Open Access funding enabled and organized by Projekt DEAL. The research was partially supported by the German Science Foundation (DFG) under contract ME 3825/1.

Declarations

Conflict of interest The authors declare no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Angelopoulos, S., Dürr, C., Jin, S.: Online maximum matching with recourse. In: MFCS, volume 117 of LIPIcs, pp. 8:1–8:15 (2018)
2. Antoniadis, A., Fischer, C., Tönnis, A.: A collection of lower bounds for online matching on the line. In: LATIN, volume 10807 of LNCS, pp. 52–65 (2018)

3. Bansal, N., Buchbinder, N., Gupta, A., Naor, J.: A randomized $O(\log^2 k)$ -competitive algorithm for metric bipartite matching. *Algorithmica* **68**(2), 390–403 (2014)
4. Bernstein, A., Dudeja, A.: Online matching with recourse: random edge arrivals. In: FSTTCS, volume 182 of LIPIcs, pp. 11:1–11:16 (2020)
5. Bernstein, A., Holm, J., Rotenberg, E.: Online bipartite matching with amortized $O(\log^2 n)$ replacements. *J. ACM* **66**(5), 37:1–37:23 (2019)
6. Bosek, B., Leniowski, D., Sankowski, P., Zych, A.: Online bipartite matching in offline time. In: FOCS, pp. 384–393 (2014)
7. Chaudhuri, K., Daskalakis, C., Kleinberg, R., Lin, H.: Online bipartite perfect matching with augmentations. In: INFOCOM, pp. 1044–1052 (2009)
8. Duan, R., Pettie, S.: Linear-time approximation for maximum weight matching. *J. ACM* **61**(1), 1:1–1:23 (2014)
9. Gairing, M., Klimm, M.: Greedy metric minimum online matchings with random arrivals. *Oper. Res. Lett.* **47**(2), 88–91 (2019)
10. Grove, E., Kao, M., Krishnan, P., Vitter, J.: Online perfect matching and mobile computing. In: WADS, volume 955 of LNCS, pp. 194–205 (1995)
11. Gu, A., Gupta, A., Kumar, A.: The power of deferral: maintaining a constant-competitive Steiner tree online. *SIAM J. Comput.* **45**(1), 1–28 (2016)
12. Gupta, A., Guruganesh, G., Peng, B., Wajc, D.: Stochastic online metric matching. In: ICALP, volume 132 of LIPIcs, pp. 67:1–67:14 (2019)
13. Gupta, A., Kumar, A., Stein, C.: Maintaining assignments online: matching, scheduling, and flows. In: SODA, pp. 468–479 (2014)
14. Gupta, A., Lewi, K.: The online metric matching problem for doubling metrics. In: ICALP, volume 7391 of LNCS, pp. 424–435 (2012)
15. Gupta, V., Krishnaswamy, R., Sandeep, S.: Permutation strikes back: the power of recourse in online metric matching. In: APPROX-RANDOM, volume 176 of LIPIcs, pp. 40:1–40:20 (2020)
16. Imase, M., Waxman, B.: Dynamic Steiner tree problem. *SIAM J. Discrete Math.* **4**(3), 369–384 (1991)
17. Kalyanasundaram, B., Pruhs, K.: Online weighted matching. *J. Algorithms* **14**(3), 478–488 (1993)
18. Karp, R., Vazirani, U., Vazirani, V.: An optimal algorithm for on-line bipartite matching. In: STOC, pp. 352–358 (1990)
19. Khuller, S., Mitchell, S., Vazirani, V.: On-line algorithms for weighted bipartite matching and stable marriages. *Theor. Comput. Sci.* **127**(2), 255–267 (1994)
20. Kuhn, H.W., Yaw, B.: The Hungarian method for the assignment problem. *Naval Res. Logist. Quart.* **2**, 83–97 (1955)
21. Lacki, J., Oćwieja, J., Pilipczuk, M., Sankowski, P., Zych, A.: The power of dynamic distance oracles: efficient dynamic algorithms for the Steiner tree. In: STOC, pp. 11–20 (2015)
22. Matuschke, J., Schmidt-Kraepelin, U., Verschae, J.: Maintaining perfect matchings at low cost. In: ICALP, volume 132 of LIPIcs, pp. 82:1–82:14 (2019)
23. Megow, N., Nölke, L.: Online minimum cost matching with recourse on the line. In: APPROX-RANDOM, volume 176 of LIPIcs, pp. 37:1–37:16 (2020)
24. Megow, N., Skutella, M., Verschae, J., Wiese, A.: The power of recourse for online MST and TSP. *SIAM J. Comput.* **45**(3), 859–880 (2016)
25. Mehta, A.: Online matching and ad allocation. *Found. Trends Theor. Comput. Sci.* **8**(4), 265–368 (2013)
26. Nayyar, K., Raghvendra, S.: An input sensitive online algorithm for the metric bipartite matching problem. In: FOCS, pp. 505–515 (2017)
27. Peserico, E., Scquizzato, M.: Matching on the line admits no $o(\sqrt{\log n})$ -competitive algorithm. In: ICALP, volume 198 of LIPIcs, pp. 103:1–103:3 (2021)
28. Raghvendra, S.: A robust and optimal online algorithm for minimum metric bipartite matching. In: APPROX-RANDOM, volume 60 of LIPIcs, pp. 18:1–18:16 (2016)
29. Raghvendra, S.: Optimal analysis of an online algorithm for the bipartite matching problem on a line. In: SoCG, volume 99 of LIPIcs, pp. 67:1–67:14 (2018)
30. Shin, Y., Kim, K., Lee, S., An, H.: Online graph matching problems with a worst-case reassignment budget (2020). *CoRR* [arXiv:2003.05175](https://arxiv.org/abs/2003.05175)